考试中心填写：

___年___月___日
考试用

湖南大学课程考试试卷

课程名称：计算机系统　；试卷编号：A；考试时间：120 分钟

专业班级：　学号：　姓名：

题号	一	二	三	四	五	六	七	八	九	十	总分
应得分	10	15	10	15	25	25					100
实得分											
评卷人											评分：

（如无特别说明，所示汇编代码均在 32 位机小端环境下编译得到）

1.简答题（10 分）

在某机器中，int 型、unsigned int 型均占一个字节（8bit），short 型占用 4 个 bit，并且有符号数均采用补码表示。float 型也用一个字节来表示，并遵循 IEEE754 规范：最高位表示符号，中间 4 位为阶码，最后 3 位为尾数。

（1）请以此机器为例，用数字 225 来解释浮点数的“向偶数舍入”和“int 转换为 float 时，数字不会溢出，但可能被舍入”。（4 分）

（2）假设有如下代码:

```
float f= -1;
int *pi=&f;
int i=*pi;
short s=(short)i;
unsigned int ui=(unsigned int)s;
float *pf=&ui;
```

请写出 f 的二进制表示，ui 的二进制表示，*pf 的值。（6 分）

2.程序填空题（15 分，每空 3 分）

如下是一个 c 语言程序及其对应的汇编代码，请参照汇编代码，完成 c 程序的空缺部分。

c 语言程序：

```
#include <stdio.h>

int main()
{
    int i,j,k=0,loop;
    int* p;
    int arr[3][2]={{9,5},{   (1)   ,22},{6,18}};
    scanf("%d,%d",&i,&j);
    p=arr+2;
```

```
        do
        {
            if(   (2)   )
                k+=*p;
            else
                k+=  (3)   ;
        }while(  (4)   );
        return  (5)  ;
    }
```

汇编代码如下：

```
    .section  .rodata
.LC0:
    .string   "%d,%d"
main:
    pushl  %ebp
    movl   %esp, %ebp
    andl   $-16, %esp
    subl   $64, %esp
    movl   $0, 56(%esp)
    movl   $9, 24(%esp)
    movl   $5, 28(%esp)
    movl   $31, 32(%esp)
    movl   $22, 36(%esp)
    movl   $6, 40(%esp)
    movl   $18, 44(%esp)
    movl   $.LC0, %eax
    leal   52(%esp), %edx
    movl   %edx, 8(%esp)
    leal   48(%esp), %edx
    movl   %edx, 4(%esp)
    movl   %eax, (%esp)
    call   __isoc99_scanf
    leal   24(%esp), %eax
    addl   $16, %eax
    movl   %eax, 60(%esp)
.L6:
    movl   48(%esp), %edx
    movl   52(%esp), %eax
    imull  %eax, %edx
    movl   60(%esp), %eax
    movl   (%eax), %eax
    movl   %edx, %ecx
    subl   %eax, %ecx
```

```
    movl   %ecx, %eax
    cmpl   $6, %eax
    jle .L2
    movl   60(%esp), %eax
    movl   (%eax), %eax
    addl   %eax, 56(%esp)
    jmp .L3
.L2:
    movl   60(%esp), %eax
    subl   $4, %eax
    movl   (%eax), %eax
    cmpl   $2, %eax
    jg  .L4
    movl   48(%esp), %edx
    movl   52(%esp), %eax
    addl   %edx, %eax
    jmp .L5
.L4:
    movl   48(%esp), %edx
    movl   $0, %eax
    subl   %edx, %eax
    addl   %eax, %eax
    addl   56(%esp), %eax
.L5:
    addl   %eax, 56(%esp)
.L3:
    movl   48(%esp), %edx
    movl   52(%esp), %eax
    movl   %edx, %ecx
    imull  %eax, %ecx
    testl  %ecx, %ecx
    setg   %cl
    addl   $1, %edx
    movl   %edx, 48(%esp)
    subl   $1, %eax
    movl   %eax, 52(%esp)
    testb  %cl, %cl
    jne .L6
    movl   48(%esp), %eax
    cmpl   $4, %eax
    jle .L6
    movl   60(%esp), %eax
    movl   (%eax), %eax
    movl   %eax, %edx
```

```
    addl   56(%esp), %edx
    movl   48(%esp), %ecx
    movl   52(%esp), %eax
    imull  %ecx, %eax
    movl   %edx, %ecx
    subl   %eax, %ecx
    movl   %ecx, %eax
    leave
    ret
```

3.程序填空题（第一空 4 分，其余两空每空 3 分，共 10 分）

如下是一个 c 语言程序及其对应的汇编代码，请参照汇编代码，完成 c 程序的空缺部分。

c 语言程序：

```
struct s
{
    int value;
    struct s *lchild;
    struct s *rchild;
}*root,*ps,*ps1,*ps2;
int f1(struct s * ps3)
{
    if(ps3==NULL) return 0;
    return    (1)    ;
}
int f2(int m,int n)
{
    return m*m+n*n;
}
int main()
{
    int i,j,k;
    root=(struct s*)malloc(sizeof(struct s));
    root->value=2025;
    root->lchild=NULL;
    root->rchild=NULL;
    ps=root;
    scanf("%d,%d\n",&i,&j);
    while(f2(i,j)!=0)
    {
        ps1=(struct s*)malloc(sizeof(struct s));
        ps1->lchild=NULL;
        ps1->rchild=NULL;ps1->value=i;
        ps2=(struct s*)malloc(sizeof(struct s));
        ps2->lchild=NULL;
```

```
        ps2->rchild=NULL;
        ps->lchild=ps1;
        ps->rchild=ps2; ps1->value=j;
        if(   (2)   )
            ps=ps->lchild;
        else
            ps=ps->rchild;
    }
    return   (3)   ;
}
```

汇编代码如下：

```
f1:
    pushl  %ebp
    movl   %esp, %ebp
    pushl  %esi
    pushl  %ebx
    subl   $16, %esp
    cmpl   $0, 8(%ebp)
    jne .L2
    movl   $0, %eax
    jmp .L3
.L2:
    movl   8(%ebp), %eax
    movl   (%eax), %ebx
    movl   8(%ebp), %eax
    movl   8(%eax), %eax
    movl   %eax, (%esp)
    call   f1
    movl   %eax, %edx
    movl   %edx, %eax
    addl   %eax, %eax
    leal   (%eax,%edx), %esi
    movl   8(%ebp), %eax
    movl   4(%eax), %eax
    movl   %eax, (%esp)
    call   f1
    addl   %eax, %eax
```

```
    addl   %esi, %eax
    addl   %ebx, %eax
.L3:
    addl   $16, %esp
    popl   %ebx
    popl   %esi
    popl   %ebp
    ret
f2:
    pushl  %ebp
    movl   %esp, %ebp
    movl   8(%ebp), %eax
    movl   %eax, %edx
    imull  8(%ebp), %edx
    movl   12(%ebp), %eax
    imull  12(%ebp), %eax
    addl   %edx, %eax
    popl   %ebp
    ret
    .section  .rodata
.LC0:
    .string   "%d,%d\n"
main:
    pushl  %ebp
    movl   %esp, %ebp
    pushl  %ebx
    andl   $-16, %esp
    subl   $32, %esp
    movl   $12, (%esp)
    call   malloc
    movl   %eax, root
    movl   root, %eax
    movl   $2025, (%eax)
    movl   root, %eax
    movl   $0, 4(%eax)
    movl   root, %eax
    movl   $0, 8(%eax)
```

```
    movl   root, %eax
    movl   %eax, ps
    movl   $.LC0, %eax
    leal   28(%esp), %edx
    movl   %edx, 8(%esp)
    leal   24(%esp), %edx
    movl   %edx, 4(%esp)
    movl   %eax, (%esp)
    call   __isoc99_scanf
    jmp .L6
.L8:
    movl   $12, (%esp)
    call   malloc
    movl   %eax, ps1
    movl   ps1, %eax
    movl   $0, 4(%eax)
    movl   ps1, %eax
    movl   $0, 8(%eax)
    movl   ps1, %eax
    movl   24(%esp), %edx
    movl   %edx, (%eax)
    movl   $12, (%esp)
    call   malloc
    movl   %eax, ps2
    movl   ps2, %eax
    movl   $0, 4(%eax)
    movl   ps2, %eax
    movl   $0, 8(%eax)
    movl   ps2, %eax
    movl   28(%esp), %edx
    movl   %edx, (%eax)
    movl   ps, %eax
    movl   ps1, %edx
    movl   %edx, 4(%eax)
    movl   ps, %eax
    movl   ps2, %edx
    movl   %edx, 8(%eax)
```

```
    movl   24(%esp), %edx
    movl   28(%esp), %eax
    movl   %edx, 4(%esp)
    movl   %eax, (%esp)
    call   f2
    movl   24(%esp), %edx
    movl   %eax, 4(%esp)
    movl   %edx, (%esp)
    call   f2
    cmpl   $2025, %eax
    jle .L7
    movl   ps, %eax
    movl   4(%eax), %eax
    movl   %eax, ps
    jmp .L6
.L7:
    movl   ps, %eax
    movl   8(%eax), %eax
    movl   %eax, ps
.L6:
    movl   28(%esp), %edx
    movl   24(%esp), %eax
    movl   %edx, 4(%esp)
    movl   %eax, (%esp)
    call   f2
    testl  %eax, %eax
    jne .L8
    movl   root, %eax
    movl   4(%eax), %eax
    movl   %eax, (%esp)
    call   f1
    movl   %eax, %ebx
    movl   root, %eax
    movl   %eax, (%esp)
    call   f1
    movl   %ebx, 4(%esp)
    movl   %eax, (%esp)
```

```
    call   f2
    movl   -4(%ebp), %ebx
    leave
    ret
```

4.综合分析题（15 分）

一个简单的函数调用 C 代码经编译后得到的核心汇编代码如下：

```
0804841d <main>:
 804841d: 55                    push   %ebp
 804841e: 89 e5                 mov    %esp,%ebp
 8048420: 83 e4 f0              and    $0xfffffff0,%esp
 8048423: 83 ec 30              sub    $0x30,%esp
 8048426: c7 44 24 1c 46 00 00    movl   $0x46,0x1c(%esp)
 804842d: 00
 804842e: c7 44 24 20 09 00 00    movl   $0x9,0x20(%esp)
 8048435: 00
 8048436: c7 44 24 24 00 00 00    movl   $0x0,0x24(%esp)
 804843d: 00
 804843e: 8d 44 24 1c           lea    0x1c(%esp),%eax
 8048442: 89 44 24 28           mov    %eax,0x28(%esp)
 8048446: 8d 44 24 20           lea    0x20(%esp),%eax
 804844a: 89 44 24 2c           mov    %eax,0x2c(%esp)
 804844e: 8b 44 24 2c           mov    0x2c(%esp),%eax
 8048452: 89 44 24 04           mov    %eax,0x4(%esp)
 8048456: 8b 44 24 28           mov    0x28(%esp),%eax
 804845a: 89 04 24              mov    %eax,(%esp)
 804845d: e8 1a 00 00 00        call   804847c <f1>
 8048462: 89 44 24 24           mov    %eax,0x24(%esp)
 8048466: 8b 44 24 24           mov    0x24(%esp),%eax
 804846a: 89 44 24 04           mov    %eax,0x4(%esp)
 804846e: c7 04 24 40 85 04 08    movl  $0x8048540,(%esp)
 8048475: e8 76 fe ff ff        call 80482f0 <printf@plt>
 804847a: c9                    leave
 804847b: c3                    ret

0804847c <f1>:
 804847c: 55                    push   %ebp
 804847d: 89 e5                 mov    %esp,%ebp
 804847f: 83 ec 10              sub    $0x10,%esp
 8048482: c7 45 fc 00 00 00 00    movl   $0x0,-0x4(%ebp)
 8048489: 8b 45 08              mov    0x8(%ebp),%eax
 804848c: 8b 10                 mov    (%eax),%edx
 804848e: 89 d0                 mov    %edx,%eax
```

```
8048490: c1 e0 02            shl    $0x2,%eax
8048493: 01 c2                add    %eax,%edx
8048495: 8b 45 0c             mov    0xc(%ebp),%eax
8048498: 8b 00                mov    (%eax),%eax
804849a: 01 d0                add    %edx,%eax
804849c: 01 c0                add    %eax,%eax
804849e: 89 45 fc             mov    %eax,-0x4(%ebp)
80484a1: 8b 45 fc             mov    -0x4(%ebp),%eax
80484a4: c9                   leave
80484a5: c3
```

下图为该程序执行函数 f1 时的栈帧图，main 函数运行时 **ESP=0xBFFFF0C0**。**该机器上局部变量遵循“先声明者栈中地址小”的存储规则**。请在栈帧图“？”处填上正确的数值。**注意：此时 f1 刚执行完毕，尚未将计算结果返回主函数！**

	地址	内容
main 函数起始点	0xBFFFF0F8	OLD EBP
	……	……
	0EC	？（1 分）
	0E8	？（1 分）
	0E4	？（1 分）
	0E0	？（1 分）
	0DC	？（1 分）
	……	……
	0C4	？（2 分）
	0C0	？（2 分）
	0BC	？（3 分）
	0B8	0xBFFFF0F8
	0B4	？（3 分）
	……	……
f1 函数栈帧顶部	(ESP) -> 0A8	

函数调用栈帧图

5.分析题（25 分）

在 linux 中，有如下四个 C 语言代码文件组成的项目如下，请阅读代码并回答如下问题。

1） 请分析， main.c 文件如何生成相应的 main.o 文件，这个.o 文件的基本格式（即大致分成几部分内容）是怎样，其中包含哪些符号？强弱属性如何？ （6 分）

2） 请分别描述 对 global_factor 和 compute_sum_squares 这两个符号引用，如何进行重定位？（4 分）

3） 如果要使用 gcc 静态链接前述几个 C 文件，形成可执行文件 test，应该在命令行输入什么样的命令？其运行结果是什么？（5 分）

4)在 main.c 文件中对于 SIGCHLD 信号处理会出现什么问题？为什么会出现这样的问题？怎么解决？（5 分）

5） 对于 compute.c 文件中的代码，利用本课程优化的知识，为了提高指令级并行提高性能，如何进行优化？（5 分）

```c
/* compute.c */
extern int global_factor;  // 在其他文件中定义的全局变量
// 计算数组元素的加权平方和
int compute_sum_squares(int *array, int size) {
    int sum = 0;
    for (int i = 0; i < size; i++) {
        sum += array[i] * array[i] * global_factor;}
    return sum;  }
```

```c
/* globals.c */
int global_factor = 2;  // 全局变量定义
```

```c
/* child_handler.c */
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>

void handle_child_exit() {
    int status;
    pid_t pid;

    pid = wait(&status);
    if (pid > 0) {
        printf("Child %d exited with status %d\n", pid,
WEXITSTATUS(status));
    }
```

```
/* main.c */
#include <stdio.h>
#include <stdlib.h>
 #include <unistd.h>
#include <signal.h>
#define ARRAY_SIZE 1000
#define NUM_CHILDREN 3

// 函数与变量声明
 extern int compute_sum_squares(int *array, int size);
 extern int global_factor;
 extern void handle_child_exit();

void sigchld_handler(int sig) {
     printf("Received SIGCHLD signal\n");
     handle_child_exit(); }

int main() {
   signal(SIGCHLD, sigchld_handler);

   int *data = malloc(ARRAY_SIZE * sizeof(int));
   for (int i = 0; i < ARRAY_SIZE; i++) { data[i] = i % 100; }

    global_factor = 3;

    for (int i = 0; i < NUM_CHILDREN; i++) {
      pid_t pid = fork();
      if (pid == 0) {
         int result = compute_sum_squares(data, ARRAY_SIZE);
         printf("Child %d: Sum of squares: %d\n", getpid(),
    result);
         exit(i); }
       else { /
          printf("Created child process with PID: %d\n", pid);
          usleep(10000); //挂起 10000 微秒，确保子进程启动} }
       sleep(3);
       free(data);
       printf("Parent process exiting\n");
       return 0; }
```

6．虚拟存储器地址转换（25 分）

假设一个使用虚拟存储器的计算机系统具有以下参数：

- 内存是字节可寻址的，内存访问是1字节（不是4字节）字。
- 虚拟地址为20位宽。
- 物理地址为16位宽。
- 页大小为256字节。
- TLB有4组，4路组相联。
- 高速缓存有32行，是2路组相联，具有4字节行大小。

在下表中所有数字均以十六进制表示。TLB、部分页表和高速缓存的内容如下：

表 1 TLB

Index	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid
0x0	0x1A3	0x3D	1	0x2B1	0x5E	1	0x3A2	0x9D	0	0x4C9	0x15	1
0x1	0x07A	0x8C	1	0x2F3	0xE2	1	0x3D2	0x26	1	0x4E1	0x3F	0
0x2	0x1C7	0x4E	1	0x2E1	0xC0	0	0x3C6	0xBF	1	0x4D8	0x60	1
0x3	0x1F5	0x7B	1	0x2D4	0xAE	1	0x3B7	0x04	1	0x4B5	0x4D	1

表 2　页表

VPN	PPN	Valid	VPN	PPN	Valid
0x1A3	0x3D	1	0x2B1	0x5E	1
0x1C7	0x4E	1	0x2F5	0x7B	1
0x1E9	0x8C	1	0x3A2	0x9D	0
0x2D4	0xAE	1	0x3C6	0xBF	1
0x2E1	0xC0	0	0x3F8	0xD1	1
0x2F3	0xE2	1	0x4A5	0xF3	0
0x3B7	0x04	1	0x4C9	0x15	1
0x3D2	0x26	1	0x4E1	0x3F	0
0x4B5	0x4D	1	0x5A3	0x6A	0
0x4D8	0x60	1	0x5C7	0x82	0
0x5E2	0x93	1	0x5F1	0xB5	0
0x5F6	0xC6	1	0x6A4	0xD7	0
0x6B3	0xE8	1	0x6C5	0xF9	1
0x7D1	0x1A	1	0x7E4	0x2B	1
0x7F2	0x0C	0	0x8A6	0x4E	1
0x8C9	0x5F	0	0x8E3	0x71	1

表 3 高速缓存

Index	Tag	Valid	line	Tag	Valid	line
0x0	0x13B	1	11 22 33 44	0x2B1	1	55 66 77 88
0x1	0x1E9	1	AA BB CC DD	0x2F3	1	EE FF 00 11
0x2	0x1C7	1	33 44 55 66	0x2E1	0	00 00 00 00
0x3	0x1F5	1	77 66 55 44	0x2D4	1	99 88 77 66
0x4	0x3A2	0	00 00 00 00	0x3C6	1	DD EE FF 00
0x5	0x3D2	1	44 55 66 77	0x3B7	1	11 22 33 44
0x6	0x4C9	1	22 33 44 55	0x4D8	1	66 77 88 99
0x7	0x5E2	1	88 99 AA BB	0x5F6	1	CC DD EE FF
0x8	0x6B3	1	12 34 56 78	0x6C5	1	9A BC DE F0
0x9	0x7D1	1	AB CD EF 01	0x232	0	23 45 67 89
0xA	0x123	0	00 00 00 00	0x456	1	CC DD EE FF
0xB	0x8A6	1	34 56 78 9A	0x8E3	1	BC DE F0 12
0xC	0x9F2	0	00 00 00 00	0xA34	1	45 67 89 AB
0xD	0xB56	1	CD EF 01 23	0xC78	0	00 00 00 00
0xE	0xD9A	1	56 78 9A BC	0xEBC	1	DE F0 12 34
0xF	0xFDE	1	EF 01 23 45	0x123	0	00 00 00 00

第一部分：基本参数（4.5 分）

1、指出以下字段分别在 20 位虚拟地址的哪些位（最低位为 0，用[X:Y]的区间形式表示）

VPO：_____

VPN：_____

TLBI：_____

TLBT：_____

2、指出以下字段分别在 16 位物理地址的哪些位（最低位为 0，用[X:Y]的区间形式表示）

PPO：_____

PPN：_____

CO：_____

CI：_____

CT：_____

第二部分：地址翻译（20.5 分）

对于给定的虚拟地址，请指明访问的 TLB 条目和物理地址和以十六进制返回的高速缓存字节值。指明 TLB 是否未命中、是否发生页面错误以及是否发生高速缓存未命中。

如果高速缓存未命中，请输入“N/A”作为“返回的高速缓存字节”。如果存在缺页，请在“PPN”中输入“N/A”，并将 C 部分和 D 部分留空。

虚拟地址 1：0x1E9A7

A、虚拟地址（每格一位，填写 0 或 1，此步不计分）

19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

B、地址翻译（答案以十六进制表示）

参数	值	参数	值
VPN	0x	TLB 命中？（是/否）	
TLBI	0x	缺页？（是/否）	
TLBT	0x	PPN	0x

C、**物理地址**：（每格一位，填写 0 或 1）

15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

D、物理内存引用（答案以十六进制表示）

参数	值	参数	值
CO	0x	物理地址	0x
CI	0x	高速缓存命中？（是/否）	
CT	0x	高速缓存返回的字节值	0x

虚拟地址 2：0x1C7C3

A、虚拟地址（每格一位，填写 0 或 1，此步不计分）

19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

B、地址翻译（答案以十六进制表示）

参数	值	参数	值
VPN	0x	TLB 命中？（是/否）	
TLBI	0x	缺页？（是/否）	
TLBT	0x	PPN	0x

C、**物理地址**：（每格一位，填写 0 或 1）

15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

D、物理内存引用（答案以十六进制表示）

参数	值	参数	值
CO	0x	物理地址	0x
CI	0x	高速缓存命中？（是/否）	
CT	0x	高速缓存返回的字节值	0x